

AWS Base Module Documentation

Business-style build record for AWS account preparation, SES/S3/SQS setup, environment configuration, dev email fallback, queue helpers, and final verification.

TIME BLOCK

11:45 AM - 12:15 PM

MODULE

AWS Base

STATUS

Complete

Objective

Prepare the platform for AWS-backed email sending, asset storage, and queue processing while keeping the application safe for local development through `EMAIL_PROVIDER=dev`.

Why This Module Matters

SES Ready

The mailer can send through AWS SES later, but logs safely in dev mode today.

SQS Ready

The app has queue helpers for campaign sending and future worker processing.

S3 Ready

The app has an upload helper for future email images and template assets.

AWS Console Setup Completed

Service	Setup	Purpose
IAM	Created/used development access credentials and attached email/storage/queue permissions.	Allows local code to call AWS services.
SES	Verified sender email in Asia Pacific Hyderabad.	Required before SES can send mail from that address.

SES Configuration Set	email-platform-events	Future open, click, delivery, bounce, and complaint event tracking.
S3	email-platform-assets-shirisha-2026	Future storage for template images and assets.
SQS	email-platform-send-queue and email-platform-events-queue	Future background email sending and SES event processing.
Region	ap-south-2 Hyderabad	All AWS resources and code config use the same region.

Security Note

AWS keys were pasted during setup. They are redacted in this documentation. Rotate/delete that IAM access key in AWS IAM and create a fresh key before deploying or committing anything.

Environment Configuration

The final environment targets AWS Hyderabad and keeps sending in development mode.

```
DATABASE_URL="postgres://postgres:postgres@localhost:5433/email_campaign_platform?schema=public"

JWT_ACCESS_SECRET="change-me-access-secret"
JWT_REFRESH_SECRET="change-me-refresh-secret"
ACCESS_TOKEN_MINUTES="15"
REFRESH_TOKEN_DAYS="7"

API_PORT="4000"
WEB_ORIGIN="http://localhost:5173"

AWS_REGION="ap-south-2"
AWS_ACCESS_KEY_ID="REDACTED_ROTATE_KEY"
AWS_SECRET_ACCESS_KEY="REDACTED_ROTATE_SECRET"
SES_CONFIGURATION_SET="email-platform-events"
SES_FROM_EMAIL="manapuramshiri17@gmail.com"
S3_BUCKET="email-platform-assets-shirisha-2026"

SQS_SEND_QUEUE_URL="https://sqs.ap-south-2.amazonaws.com/316704942193/email-platform-send-queue"
SQS_EVENTS_QUEUE_URL="https://sqs.ap-south-2.amazonaws.com/316704942193/email-platform-events-queue"

EMAIL_PROVIDER="dev"
```

Code Implemented

File	Purpose
apps/api/src/config/env.ts	Validates required database, JWT, AWS, SES, S3, SQS, and provider variables.
apps/api/src/services/mailer.ts	Provides dev email logging and future SESv2 email sending with unsubscribe headers.
apps/api/src/services/storage.ts	Uploads future email assets to S3.
apps/api/src/services/queue.ts	Logs queue jobs in dev mode and can send jobs to SQS later.
apps/worker/src/config/env.ts	Validates worker-side database and AWS environment variables.
apps/worker/src/services/queue.ts	Reads and deletes SQS send jobs when real queue mode is enabled.
apps/api/src/index.ts	Added temporary dev routes for testing email and queue integrations.

Dev Test Routes

Route	Result
POST /dev/test-email	Returned { "provider": "dev", "messageId": "dev-..." }
POST /dev/test-queue	Expected to return { "provider": "dev", "messageId": "dev-queue-..." } while EMAIL_PROVIDER=dev.

Commands Run

```
pnpm --filter api add @aws-sdk/client-sesv2 @aws-sdk/client-s3 @aws-sdk/client-sqs
pnpm --filter worker add @aws-sdk/client-sesv2 @aws-sdk/client-sqs
pnpm --filter worker add zod
pnpm --filter worker add -D dotenv-cli
pnpm --filter api dev
curl -X POST http://localhost:4000/dev/test-email
pnpm --filter api typecheck
pnpm --filter api build
pnpm typecheck
pnpm build
```

Successful Outputs

Dev Email Test

```
{"provider":"dev","messageId":"dev-1778396911069"}
```

Root Typecheck

```
apps/api typecheck completed  
apps/web typecheck completed  
apps/worker typecheck completed
```

Root Build

```
apps/api build completed  
apps/web build completed  
apps/worker build completed
```

Issues Found and Fixed

Worker missing zod: worker env validation imported zod before it was installed. Fixed by adding zod to worker dependencies.

API env not loading: API dev command originally ran `tsx watch src/index.ts` directly. Fixed by loading root `.env` through `dotenv-cli`.

Worker env loading: worker scripts were updated to load root `.env` consistently.

Filter confusion: `direct --filter worker` did not match, but root typecheck/build confirmed worker compilation succeeded.

Completion Status

AWS Base Module Complete

AWS account resources are prepared, local environment is configured, AWS SDK packages are installed, API/worker helpers are implemented, dev email route works, and the monorepo passes typecheck and build.

Next Module

M1 Login & User Management: implement login, JWT auth, refresh handling, current user endpoint, role middleware, and profile/user-management foundations.

Code Appendix

apps/api/package.json

```
{
  "name": "api",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "main": "dist/index.js",
  "scripts": {
    "dev": "dotenv -e ../../.env -- tsx watch src/index.ts",
    "build": "tsc -p tsconfig.json",
    "start": "node dist/index.js",
    "typecheck": "tsc -p tsconfig.json --noEmit",
    "lint": "echo \"lint api\"",
    "prisma:generate": "dotenv -e ../../.env -- prisma generate --schema ../../prisma/schema.prisma",
    "prisma:migrate": "dotenv -e ../../.env -- prisma migrate dev --schema ../../prisma/schema.prisma",
    "prisma:seed": "dotenv -e ../../.env -- tsx src/seed.ts",
    "prisma:studio": "dotenv -e ../../.env -- prisma studio --schema ../../prisma/schema.prisma"
  },
  "dependencies": {
    "@aws-sdk/client-s3": "^3.1045.0",
    "@aws-sdk/client-sesv2": "^3.1045.0",
    "@aws-sdk/client-sqs": "^3.1045.0",
    "@prisma/client": "^6.19.3",
    "bcryptjs": "^3.0.2",
    "cors": "^2.8.5",
    "dotenv": "^16.5.0",
    "express": "^5.1.0",
    "jsonwebtoken": "^9.0.2",
    "zod": "^3.24.4"
  },
  "devDependencies": {
    "@types/bcryptjs": "^2.4.6",
    "@types/cors": "^2.8.17",
    "@types/express": "^5.0.1",
    "@types/jsonwebtoken": "^9.0.9",
    "@types/node": "^22.15.17",
    "dotenv-cli": "^7.4.4",
    "prisma": "^6.19.3",
    "tsx": "^4.19.4",
    "typescript": "^5.8.3"
  }
}
```

apps/api/src/config/env.ts

```
import { z } from "zod";

const envSchema = z.object({
  DATABASE_URL: z.string().min(1),

  JWT_ACCESS_SECRET: z.string().min(1),
  JWT_REFRESH_SECRET: z.string().min(1),
});
```

```

ACCESS_TOKEN_MINUTES: z.coerce.number().default(15),
REFRESH_TOKEN_DAYS: z.coerce.number().default(7),

API_PORT: z.coerce.number().default(4000),
WEB_ORIGIN: z.string().url().default("http://localhost:5173"),

AWS_REGION: z.string().default("ap-south-2"),
AWS_ACCESS_KEY_ID: z.string().optional(),
AWS_SECRET_ACCESS_KEY: z.string().optional(),
SES_CONFIGURATION_SET: z.string().default("email-platform-events"),
SES_FROM_EMAIL: z.string().email(),
S3_BUCKET: z.string().optional(),
SQS_SEND_QUEUE_URL: z.string().optional(),
SQS_EVENTS_QUEUE_URL: z.string().optional(),
EMAIL_PROVIDER: z.enum(["dev", "ses"]).default("dev")
});

export const env = envSchema.parse(process.env);

```

apps/api/src/services/mailer.ts

```

import { SESv2Client, SendEmailCommand } from "@aws-sdk/client-sesv2";
import { env } from "../../config/env.js";

type SendEmailInput = {
  to: string;
  subject: string;
  html: string;
  text?: string;
  from?: string;
  replyTo?: string;
  unsubscribeUrl?: string;
};

const ses = new SESv2Client({
  region: env.AWS_REGION
});

export async function sendEmail(input: SendEmailInput) {
  const fromEmail = input.from ?? env.SES_FROM_EMAIL;

  if (env.EMAIL_PROVIDER === "dev") {
    console.log("[DEV EMAIL]");
    console.log("To:", input.to);
    console.log("Subject:", input.subject);
    console.log("From:", fromEmail);
    console.log("Reply-To:", input.replyTo ?? "-");
    console.log("Unsubscribe:", input.unsubscribeUrl ?? "-");
    console.log("HTML:", input.html.slice(0, 300));

    return {
      provider: "dev",
      messageId: `dev-${Date.now()}`
    };
  }

  const headers = input.unsubscribeUrl
    ? [
        {
          Name: "List-Unsubscribe",

```

```

        Value: `<${input.unsubscribeUrl}>`
      },
      {
        Name: "List-Unsubscribe-Post",
        Value: "List-Unsubscribe=One-Click"
      }
    ]
  : undefined;

const result = await ses.send(
  new SendEmailCommand({
    FromEmailAddress: fromEmail,
    Destination: {
      ToAddresses: [input.to]
    },
    ReplyToAddresses: input.replyTo ? [input.replyTo] : undefined,
    ConfigurationSetName: env.SES_CONFIGURATION_SET,
    Content: {
      Simple: {
        Subject: {
          Data: input.subject,
          Charset: "UTF-8"
        },
        Body: {
          Html: {
            Data: input.html,
            Charset: "UTF-8"
          },
          Text: input.text
            ? {
                Data: input.text,
                Charset: "UTF-8"
              }
            : undefined
        },
        Headers: headers
      }
    }
  })
);

return {
  provider: "ses",
  messageId: result.MessageId
};
}

```

apps/api/src/services/storage.ts

```

import { PutObjectCommand, S3Client } from "@aws-sdk/client-s3";
import { env } from "../../config/env.js";

const s3 = new S3Client({
  region: env.AWS_REGION
});

type UploadAssetInput = {
  key: string;
  body: Buffer;
  contentType: string;
}

```



```

};

export async function uploadAsset(input: UploadAssetInput) {
  if (!env.S3_BUCKET) {
    throw new Error("S3_BUCKET is not configured.");
  }

  await s3.send(
    new PutObjectCommand({
      Bucket: env.S3_BUCKET,
      Key: input.key,
      Body: input.body,
      ContentType: input.contentType
    })
  );

  return {
    bucket: env.S3_BUCKET,
    key: input.key,
    url: `https://${env.S3_BUCKET}.s3.${env.AWS_REGION}.amazonaws.com/${input.key}`
  };
}

```

apps/api/src/services/queue.ts

```

import { SendMessageCommand, SQSClient } from "@aws-sdk/client-sqs";
import { env } from "../../config/env.js";

const sqs = new SQSClient({
  region: env.AWS_REGION
});

export async function enqueueSendJob(payload: unknown) {
  if (!env.SQS_SEND_QUEUE_URL) {
    console.log("[DEV QUEUE] send job");
    console.log(JSON.stringify(payload, null, 2));

    return {
      provider: "dev",
      messageId: `dev-queue-${Date.now()}`
    };
  }

  if (env.EMAIL_PROVIDER === "dev") {
    console.log("[DEV QUEUE] send job");
    console.log(JSON.stringify(payload, null, 2));

    return {
      provider: "dev",
      messageId: `dev-queue-${Date.now()}`
    };
  }

  const result = await sqs.send(
    new SendMessageCommand({
      QueueUrl: env.SQS_SEND_QUEUE_URL,
      MessageBody: JSON.stringify(payload)
    })
  );
}

```

```
    return {
      provider: "sqs",
      messageId: result.MessageId
    };
  }
}
```

apps/api/src/index.ts

```
import "dotenv/config";
import express from "express";
import cors from "cors";
import { sendEmail } from "../services/mailer.js";
import { enqueueSendJob } from "../services/queue.js";

const app = express();

const port = Number(process.env.API_PORT ?? 4000);
const webOrigin = process.env.WEB_ORIGIN ?? "http://localhost:5173";

app.use(cors({ origin: webOrigin }));
app.use(express.json());

app.get("/", (_req, res) => {
  res.json({
    message: "Email Campaign Platform API",
    health: "/health"
  });
});

app.get("/health", (_req, res) => {
  res.json({
    status: "ok",
    service: "api"
  });
});

app.post("/dev/test-email", async (_req, res) => {
  const result = await sendEmail({
    to: "test@example.com",
    subject: "Dev email test",
    html: "<h1>Hello from dev mailer</h1>",
    unsubscribeUrl: "http://localhost:4000/unsubscribe?uid=dev-test"
  });

  res.json(result);
});

app.post("/dev/test-queue", async (_req, res) => {
  const result = await enqueueSendJob({
    type: "SEND_EMAIL",
    campaignId: "dev-campaign",
    contactId: "dev-contact"
  });

  res.json(result);
});

app.listen(port, () => {
```

```
console.log(`API running on http://localhost:${port}`);
});
```

apps/worker/src/config/env.ts

```
import { z } from "zod";

const envSchema = z.object({
  DATABASE_URL: z.string().min(1),

  AWS_REGION: z.string().default("ap-south-2"),
  AWS_ACCESS_KEY_ID: z.string().optional(),
  AWS_SECRET_ACCESS_KEY: z.string().optional(),
  SES_CONFIGURATION_SET: z.string().default("email-platform-events"),
  SES_FROM_EMAIL: z.string().email(),
  SQS_SEND_QUEUE_URL: z.string().optional(),
  SQS_EVENTS_QUEUE_URL: z.string().optional(),
  EMAIL_PROVIDER: z.enum(["dev", "ses"]).default("dev")
});

export const env = envSchema.parse(process.env);
```

apps/worker/src/services/queue.ts

```
import {
  DeleteMessageCommand,
  ReceiveMessageCommand,
  SQSClient
} from "@aws-sdk/client-sqs";
import { env } from "../../config/env.js";

const sqs = new SQSClient({
  region: env.AWS_REGION
});

export async function receiveSendJobs() {
  if (!env.SQS_SEND_QUEUE_URL || env.EMAIL_PROVIDER === "dev") {
    console.log("[DEV WORKER] no real SQS polling in dev mode");
    return [];
  }

  const result = await sqs.send(
    new ReceiveMessageCommand({
      QueueUrl: env.SQS_SEND_QUEUE_URL,
      MaxNumberOfMessages: 5,
      WaitTimeSeconds: 5
    })
  );

  return result.Messages ?? [];
}

export async function deleteSendJob(receiptHandle: string) {
  if (!env.SQS_SEND_QUEUE_URL || env.EMAIL_PROVIDER === "dev") {
    return;
  }

  await sqs.send(
```

```
new DeleteMessageCommand({
  QueueUrl: env.SQS_SEND_QUEUE_URL,
  ReceiptHandle: receiptHandle
})
);
}
```